

Domain-Specific RAG Chatbot

Project Report

1. Introduction

The Domain-Specific RAG Chatbot is a question-answering system designed for the Education domain, with a focus on Artificial Intelligence.

The system allows users to upload PDF documents and ask questions about their content.

Retrieval-Augmented Generation (RAG) is used to retrieve relevant information from the documents before generating an answer.

2. Objective

The main objective is to build a chatbot that provides answers based on uploaded documents rather than relying only on the general knowledge of a language model.

The system also provides the source document and page number for retrieved information.

3. Domain

****Education → Artificial Intelligence (AI)****

The project uses educational material related to MIT 6.034 Artificial Intelligence.

4. Technologies Used

- Python
- Streamlit
- PyPDF
- LangChain Text Splitters
- Sentence Transformers
- FAISS
- Groq
- python-dotenv

5. System Workflow

The system follows these steps:

1. The user uploads PDF documents.
2. The PDF files are validated.
3. Text is extracted from each page.
4. The extracted text is divided into smaller chunks.
5. Sentence Transformer generates embeddings for the chunks.
6. The embeddings are stored in a FAISS vector database.
7. The user enters a question.
8. The question is converted into an embedding.
9. FAISS retrieves the most relevant document chunks.
10. The retrieved chunks are provided as context to the language model.
11. The language model generates an answer using the retrieved context.
12. The application displays the answer along with source document and page information.

6. RAG Components

Document Loading

`document\_loader.py` extracts text from PDF pages and stores the document name and page number as metadata.

Chunking

`rag\_pipeline.py` uses the Recursive Character Text Splitter.

The current configuration uses:

- Chunk size: 800 characters
- Chunk overlap: 120 characters

Embeddings

The project uses the pretrained:

`all-MiniLM-L6-v2`

Sentence Transformer model.

The generated embedding dimension is 384.

Vector Database

FAISS is used to store and search the document embeddings.

Cosine similarity is implemented using normalized embeddings and FAISS inner-product search.

Language Model

The project uses the Groq API with:

`openai/gpt-oss-20b`

The model receives the retrieved document context and the user's question.

7. Grounded Answering

The chatbot prompt instructs the language model to:

- Use only the retrieved context.
- Not use general knowledge.
- Not invent or guess information.
- Refuse to answer when the information is unavailable.

The refusal message is:

I could not find this information in the uploaded documents.

8. User Interface

The application uses Streamlit.

The interface provides:

- PDF upload
- File validation
- Document processing
- Question input
- Generated answer
- Source document
- Source page number
- Similarity score

9. Testing

The project includes a testing sheet containing 15 questions.

Testing includes:

- Questions whose answers are present in the documents.
- Questions requiring information from the uploaded documents.
- Questions whose answers are not present in the documents.

The testing confirms that the chatbot can retrieve relevant information and refuse questions when the required information is unavailable.

10. Security

The Groq API key is stored in the ``.env`` file.

The ``.env`` file is excluded from Git using ``.gitignore``.

The project should not be used with confidential documents without appropriate permission.

11. Limitations

The quality of the chatbot depends on the quality and content of the uploaded documents.

Retrieval errors can occur when the relevant information is not retrieved from the vector database.

The chatbot should not be assumed to be automatically correct for high-stakes information.

12. Future Improvements

Possible improvements include:

- Better retrieval techniques
- More comprehensive evaluation
- Conversation history
- Improved user interface
- More domain-specific documents
- Automated testing
- Retrieval and answer-quality metrics

13. Conclusion

The project demonstrates a complete Retrieval-Augmented Generation pipeline for a domain-specific chatbot.

It combines document processing, text chunking, embeddings, vector search, retrieval, grounded prompting, and language-model generation in a Streamlit application.